Lab 04 - Counting Sort (Part 1)

Instruction:

- This lab is divided into two parts. The objective of this part (Part 1) is to define a variation of the counting sort that operates on integers in any interval with a bounded range while maintaining linear running time. The procedures of this lab are to:
  - Define an algorithm that finds the minimum and maximum values of an array.
  - Define a variation of the counting sort.
  - Derive the running-time function of the variation of the counting sort and its helper functions.
  - Test the variation of the counting sort on a randomly generated integer array.
- Submit one `.h` file, one `.txt` file, and one `.cpp` source file.
- Your files may include only the following libraries: `iostream`, `sstream`, `string`, `vector`, `ctime`, `cctype`, and `cstdlib`. No others are permitted.
- Submit your files to your GitHub repository, inside a directory named `Lab04`.
- Cheating of any kind is prohibited and will not be tolerated.
- **Violating or failing to follow any of these rules will result in an automatic zero (0) for the lab.**

At the very beginning of your submission, add a comment containing your name and the date, to confirm you have read and understood the instructions above.

Grading

Task	Maximum Points	Points Earned
1	0.5	
2	0.5	
3	0.5	
4	0.5	
5	0.5	
Total	2.5	

Task 1

In the header file, within the namespace *aal*, define the function:

```
void Range(const vector<int>& v, int& lo, int& hi)
```

that assigns the minimum and maximum values of v to lo and hi, respectively. Assume that v is not empty.

Task 2

In the header file, within the namespace *aal*, define the function:

```
void RangedSort(const vector<int>& A, vector<int>& B)
```

that makes B a sorted copy of A by using the counting sort if the difference between the maximum and minimum values of A, plus one, does not exceed twice the length of A; otherwise, it makes B an unsorted copy of A. The function must invoke `Range()` and must shift the values of A by its minimum so that the counting sort operates correctly.

Task 3

In the text file, construct the running-time table of `Range()` and `RangedSort()`, and derive their running-time functions. Assume that each statement has a cost of 1. Show all of your work.

Task 4

In the header file, within the namespace *aal*, define the function:

```
void Print(const vector<int>& v)
```

that displays v as a list enclosed in square brackets.

Task 5

In the `main` function, verify that your `RangedSort()` function is correct by completing the following steps:

1. Declare two integer vector objects of size 20.
2. Assign the first object random values whose range does not exceed 30 and whose lower bound is not 0.
3. Display the first object by invoking `Print()`.
4. Invoke `RangedSort()` with the first object as its first argument and the second object as its second argument.
5. If the second object is sorted, display it by invoking `Print()`; otherwise, display an error message stating that the sort was unsuccessful.